
A study of Torque-Control system on UR5 for the drawing task

Jirattanon Leeudomwong, 66340500009
Bharut Aursudkit, 66340500040
Nuttapruet Puttiwarodom, 66340500018

Abstract

Industrial robot manipulators have countless applications nowadays. One of the basic tasks to test the accuracy of a robot manipulator is to perform a drawing task, which can be done with only position control alone. This project focuses on using a UR5 industrial robot manipulator to perform the drawing task. The challenge this project represents is controlling the force between the pen (end-effector) and the canvas using torque control while following a trajectory generated by the image processing method. This project is running on Gazebo Ignition physics simulation and using ROS2 as a middleware.

Keywords: Torque control, Dynamics model, Trajectory generation, Image processing, Edge detection, ROS2, Physic simulation

1 Overview

This project explore the development of a torque-controlled system for UR5e robot to draw in a simulation space. Rather than using just position controle, this project focuses on simultaneously controlling both the motion of the robot and the contact force created by the pen onto the canvas. The system integrates trajectory generation with analytic inverse kinematics, differential kinematics and dynamics based torque control. All of the components will be implemented in Gazebo with ROS2 as the communication backbone.

This report consists of six main sections, excluding the Overview:

- **Software Architecture and Simulation**

Describes the simulation environment, ROS2 ecosystem, and node-topic communication structure. Introduces the custom nodes (force-motion controller, IK node) and supporting nodes (state broadcaster, effort controller, ros-gz bridge, RViz visualization).

- **Trajectory Generation**

Explains how to generate a trajectory for drawing by using image-processing techniques. This algorithm includes covers edge detection, path extraction, and conversion into a CSV file with positions, velocities, and acceleration of end-effector for Inverse Kinematic to solve.

- **Inverse Kinematic**

Detailed into the IK method used for the UR5e via `ur_analytic_ik` library which using geometric IK decomposition which will be verify using forward kinematics and differential IK for joint velocities and accelerations is also will be compute via `pinocchio` library.

- **Dynamics and Control System**

Presents the motion control system based on UR5 dynamics. Shows the PD controller for trajectory tracking and introduces the Z-axis PI force-elevation controller to manipulate the contact force. including the full control diagram.

- **Test and Evaluation**

Discusses the experimental procedure for validating the drawing performance in simulation. By using the RMSE metric to evaluate both trajectory accuracy and force-control performance.

- **Conclusion**

Summarizes the results, highlights the effectiveness of combining torque control with force regulation, and identifies future work.

2 Software architecture and Simulation

This work performed in Gazebo ignition physic simulation. The simulation simulates the UR5, pen, gravity, collision etc. ROS2 middleware using to interface the simulation and using as the backbone of the robot system. The software architecture can be display as Node and Topic (as shown on Fig.1)

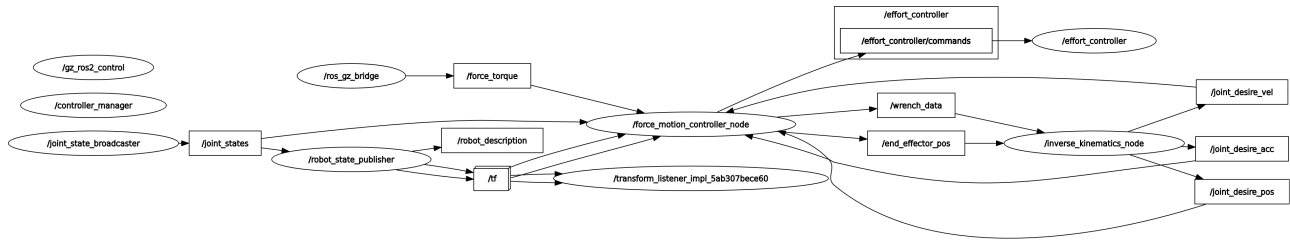


Figure 1: Nodes and topics

The `force_motion_controller_node` and `inverse_kinematic_node` were developed by the project team.

1. `force_motion_controller_node`

This node acquires and filters the wrench from the F/T sensor, reads end-effector positions, computes UR5 dynamics, and generates torques for motion control (can be found in `ur_controller` package).

2. `inverse_kinematic_node`

This node performs inverse kinematics calculations, Z-axis elevation force control and read trajectory from .csv file.

Additionally, There are several Node required to interface with the Gazebo

1. `robot_state_broadcaster`

This node publishes the current state (position, velocity) of individual joints to the `joint_states` topic

2. `robot_state_publisher`

computes and broadcasts the 3D poses of a robot's links based on its URDF model and incoming `joint_states` messages

3. `ros_gz_bridge`

The F/T sensor is instant function from Gazebo ignition. To acquire wrench from F/T sensor, only need to bridge the `gz_topic` to the ROS2 topic using `ros_gz_bridge`

4. `effort_controller`

This node use to control the UR5 joint torque. This node is part of `ROS2_control` framework

Lastly, There is addition tools helps this project display drawing line and debugging

1. `rviz2`

This program show the 3D model of the UR5, force and drawing line as show in Fig.2

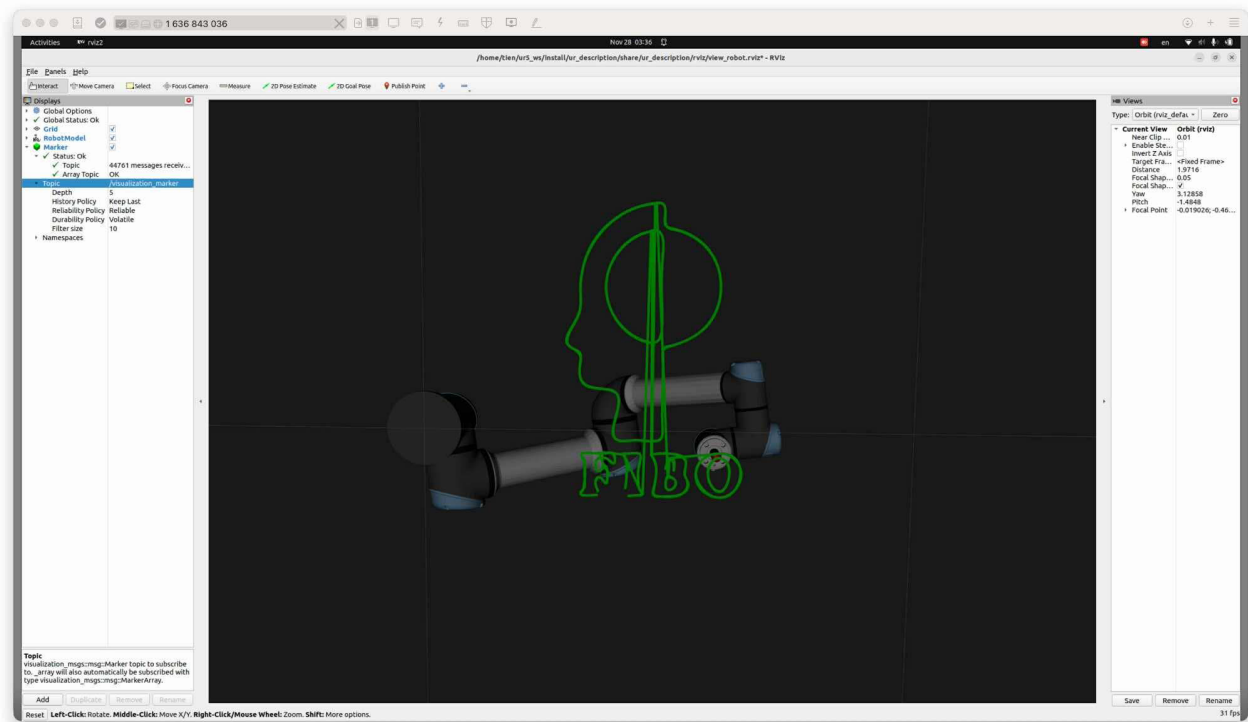


Figure 2: rviz2 interface

3 Trajectory generation

This topic will explain about the methodology of trajectory generation and result

4 Inverse Kinematic

The Inverse Kinematics (IK) used to calculate the joint angles of the UR5e robotic arm to achieve the desired end-effector position and orientation.

This section describes the process for creating both IK and differential kinematics pathways that convert desired Cartesian positions (including velocity and acceleration) into a joint space. The IK implementation can be used in an ROS2 node that publishes a target for each of the robot's joints to that robot's downstream controller.

4.1 Inverse Kinematic Method

The UR5e is a 6-DoF serial manipulator with the joint layout:

1. Base rotation
2. Shoulder
3. Elbow
4. Wrist 1
5. Wrist 2
6. Wrist 3

This structure enables a closed-form inverse kinematics solution, which is implemented in the Python library `ur_analytic_ik`.

The `ur_analytic_ik` solver uses the UR5e's modified Denavit–Hartenberg (DH) parameters which are:

Joint	a_i (m)	d_i (m)	α_i (rad)
1	0.0	0.1625	$+\pi/2$
2	-0.425	0.0	0
3	-0.3922	0.0	0
4	0.0	0.1333	$+\pi/2$
5	0.0	0.0997	$-\pi/2$
6	0.0	0.0996	0

Table 1: DH parameters of the UR5e robot.

The UR analytic IK uses classical geometric IK decomposition which is divided into 4 main steps:

1. **Solve Wrist Center Decomposition:** Compute the position of the wrist center by removing the effect of the tool extension along the end-effector z-axis:

$$\mathbf{WC} = \mathbf{P}_{EE} - d_6 \cdot \mathbf{R}_{EE} \hat{z}$$

where $\mathbf{P}_{EE}$ and $\mathbf{R}_{EE}$ are the desired end-effector position and orientation, and d_6 is the wrist to tool distance.

2. **Solve Joint 1:** Determine the base rotation angle to align the arm with the wrist center:

$$\theta_1 = \arctan 2(WC_y, WC_x)$$

3. **Solve Joints 2 and 3:** Use planar geometry and the law of cosines to compute the shoulder and elbow joint angles. Let a_2 and a_3 be the link lengths:

$$\theta_2 = \arctan 2(z', r') - \arctan 2(a_3 \sin \theta_3, a_2 + a_3 \cos \theta_3), \quad \theta_3 = \arccos \left(\frac{r'^2 - a_2^2 - a_3^2}{2a_2 a_3} \right)$$

where (r', z') is the projection of the wrist center onto the plane of joints 2 and 3.

4. **Solve Wrist Joints (4, 5, 6):** Compute the final three joint angles to achieve the desired end-effector orientation:

$${}^3R_6 = ({}^0R_3)^\top \cdot {}^0R_6$$

and the wrist angles $\theta_4, \theta_5, \theta_6$ are extracted from 3R_6 using standard Euler-angle decomposition.

And then each geometry of the manipulator and the properties of trigonometric functions give us 2 option, so the total solution for inverse kinematic is $2(\text{elbow}) \times 2(\text{shoulder}) \times 2(\text{wrist}) = 8$ possible solutions.

Then we select the best solution by filtering only solution with elbow up pose then choose the solution with the configuration closest to the previous solution.

4.2 Inverse Kinematic verification

To verify inverse kinematic we used forward kinematic that already coded in this library which is:

$${}^0T_6 = {}^0T_1 {}^1T_2 {}^2T_3 {}^3T_4 {}^4T_5 {}^5T_6$$

where each transform is:

$${}^{i-1}T_i = \begin{bmatrix} \cos \theta_i & -\sin \theta_i \cos \alpha_i & \sin \theta_i \sin \alpha_i & a_i \cos \theta_i \\ \sin \theta_i & \cos \theta_i \cos \alpha_i & -\cos \theta_i \sin \alpha_i & a_i \sin \theta_i \\ 0 & \sin \alpha_i & \cos \alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

and then compare it with the task space we used for inverse kinematic.

4.3 Differential Inverse Kinematics

To compute both joint velocities and acceleration, we need UR5e Jacobian matrix. And since `ur_analytic_ik` doesn't have Jacobian computation included. Pinocchio library is used for Jacobian computations.

4.3.1 Joint Velocities

The joint velocities are calculated from the desired end-effector velocity using the UR5e Jacobian matrix for a given joint configuration.

End-effector velocity $\mathbf{V}$ is related to the joint velocities $\dot{\mathbf{q}}$ as:

$$\mathbf{V} = J(\mathbf{q}) \dot{\mathbf{q}}$$

Rearranging gives the joint velocities:

$$\dot{\mathbf{q}} = J^\dagger \mathbf{V}$$

4.3.2 Joint Accelerations

The joint accelerations are calculated from the desired end-effector acceleration using the change in the Jacobian over time:

$$\mathbf{A} = J(\mathbf{q}) \ddot{\mathbf{q}} + \dot{J}(\mathbf{q}, \dot{\mathbf{q}}) \dot{\mathbf{q}}$$

Rearranging gives the joint accelerations:

$$\ddot{\mathbf{q}} = J^\dagger (\mathbf{A} - \dot{J} \dot{\mathbf{q}})$$

5 Dynamics and Control system

There are various type of control method available for UR5 with torque control. This work presents a motion control strategy based on the dynamics of the UR5. To generate the required torque for motion control, The inverse dynamics equation in joint space (equation 1) is used to compute required torque respect to the Joint state¹. To control position, velocity and acceleration of the UR5 using PD motion controller (equation 2)

$$\tau = M(q)\ddot{q} + C(q, \dot{q}) + g(q) \quad (1)$$

$$\ddot{q} = \ddot{q}_d + K_p(q_d - q) + K_d(\dot{q}_d - \dot{q}) \quad (2)$$

where:

- τ is the joint torque Vector
- $M(q)\ddot{q}$ is the Inertia Matrix
- $C(q, \dot{q})$ is the Coriolis and Centrifugal Matrix
- $g(q)$ is the Gravity Vector
- $\ddot{q}$ is the computed acceleration (command)
- K_p and K_d are the proportional and derivative gain matrices
- q_d and q are the desired and actual joint positions
- $\dot{q}_d$ and $\dot{q}$ are the desired and actual joint velocities
- $\ddot{q}_d$ is the desired joint acceleration

However, By using the dynamics model in joint space (equation 1) and PD controller (equation 2) are not able to control the force occurs at the end-effector. The method for control motion and force at end-effector simultaneously is to apply the Hybrid force-motion control. Even so the hybrid force-motion control is using the dynamics model in task space. Make the dynamics calculation more complicated to compute. Even if using dynamics model in joint space for motion controller and force controller together. Controller will be fighting over position. This project presents the Z-axis elevation PI control (equation 3) method to control the force occurs at the end-effector. F/T sensor is noisy sensor even apply the low-pass filter. This cause the force elevation controller is only PI controller because the D term is noise sensitive controller. This method will not affect the control system to compensate the position error with stronger force respect to the time because of the motion controller is only PD controller.

$$Z = -1 \cdot (K_p(W_d - W) + K_i \int (W_d - W) \frac{d}{dt}) \quad (3)$$

where:

- Z is the elevation in Z axis (task space)
- K_p and K_i are the proportional and integral gain matrices
- W_d and W are the desired and actual force at end-effector Z axis

After apply PD motion controller and PI elevation controller, UR5 are able to control motion and force simultaneously without fighting between 2 controllers in joint space. The control diagram is show in Fig.3. Lastly the inverse dynamics model calculation done by Pinocchio framework on python. And Controlled by the custom code written by this project team.

¹Joint state: joint position, joint velocity, joint acceleration

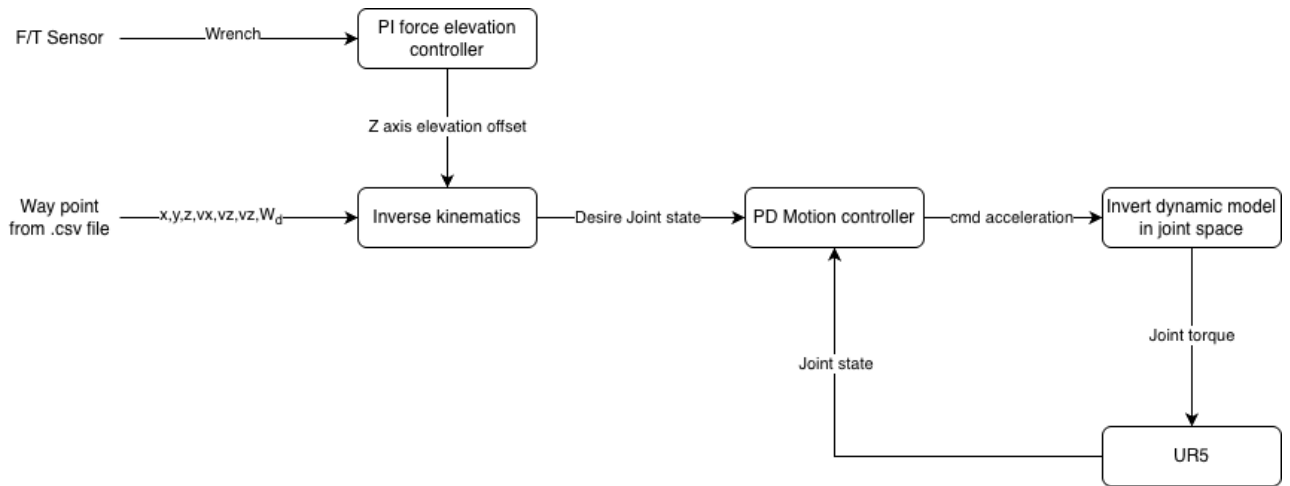


Figure 3: Control system diagram

6 Test and Evaluation

The requirement of this project is to perform drawing task within position error 10% range and able to do force control. RMSE statistic method can be used to find the average position error occurs on the system when UR5 done drawing task. With RMSE calculation code, Computer can average the RMSE of each joint over time while UR5 is operated drawing task and when UR5 is done drawing task. **The average position RMSE of this control system are 0.0022 radian or 1.9% error from joint limit.**

7 Conclusion